B.1 Assignment 03 (2025)

B.1.1 Exercise 01

1. C to RISC-V

a. Code 1:

```
# a0 = g, a1 = h
bge a1, a0, else     # do else if (h >= g)
addi a0, a0, 1       # g = g + 1
beq x0, x0, done     # jump past else block
else:
addi a1, a1, -1 # h = h - 1
done:

```

b. Code 2:

```
# a0 = g, a1 = h
blt a1, a0, else     # do else if (g > h)
addi a0, x0, 0       # g = 0
beq x0, x0, done     # jump past else block
else:
addi a1, x0, 0 # h = 0
done:

```

c. Code 3:

```
# a0 = g, a1 = h
blt a0, a1, else     # do else if (g < h)
add a0, a0, a1       # g = g + h
beq x0, x0, done     # jump past else block
else:
sub a0, a0, a1       # g = g - h
done:

```

d. Code 4:

```
# a0 = g, a1 = h
bge a0, a1, else     # do else if (g >= h)
addi a1, a1, 1       # h = h + 1
beq x0, x0, done     # jump past else block
else:
slli a1, a1, 1
done:

```

e. The C code can be implemented in RISC-V assembly as follows:

```
addi x7, x0, 0 # Init i = 0
LOOPI:
bge x7, x5, ENDI # While i < a
addi x30, x10, 0 # x30 = &D
addi x29, x0, 0 # Init j = 0
LOOPJ:
bge x29, x6, ENDJ # While j < b
add x31, x7, x29 # x31 = i+j
sw x31, 0(x30) # D[4*j] = x31
addi x30, x30, 16 # x30 = &D[4*(j+1)]
addi x29, x29, 1 # j++
jal x0, LOOPJ
ENDJ:
```

```
        addi x7, x7, 1 # i++;
        jal x0, LOOPI
        ENDI:
```

2. The code requires 12 RISC-V instructions. When a = 10 and b = 1, this results in 122 instructions being executed. It may be different for other codes.
   For example, consider the same code with different branch conditions:

```
addi x7, x0, 0 # Init i = 0
LOOPI:
bge x7, x5, ENDI # While i < a
addi x30, x10, 0 # x30 = &D
addi x29, x0, 0 # Init j = 0
LOOPJ:
bge x29, x6, ENDJ # While j < b
add x31, x7, x29 # x31 = i+j
sw x31, 0(x30) # D[4*j] = x31
addi x30, x30, 16 # x30 = &D[4*(j+1)]
addi x29, x29, 1 # j++
blt x29, x6, LOOPJ
ENDJ:
addi x7, x7, 1 # i++;
blt x7, x5, LOOPI
ENDI:
```

This results in 111 instructions being executed.

B.1.2 Exercise 02

1. x6 = 2
2. If register x6 is initialized to 10, and x5 to zero.
   a. The final value of xs is 20.
   b. C code:

```
acc = 0;
i = 10;
while (i ! = 0) {
acc += 2;
i--;
}
```

   c. 4*N + 1 instructions.
   d. (Note: change condition ! = to > = in the while loop)

```
acc = 0;
i = 10;
while (i >= 0) {
acc += 2;
i--;
}
```

3. This C code corresponds most directly to the given assembly.

```
int i;
do{
result += MemArray[i * 2];
i++;
}while (i < 100;)
return result;
```